

Exercício 28 – Troubleshooting na Pipeline de Integração Contínua

Estudo de Caso: O "Build Vermelho" na Sexta-Feira

1. A Falha no Processo Disciplinado

O principal erro cometido por Carlos foi confiar apenas na execução dos testes unitários em seu ambiente local antes de realizar o envio do código para a branch principal do repositório.

Embora os testes unitários tenham sido executados com sucesso, eles validam apenas partes isoladas da aplicação. Em um processo maduro de Integração Contínua (Continuous Integration – CI), é fundamental garantir que a alteração também seja compatível com os demais componentes do sistema, por meio da execução de testes de integração, testes funcionais e validações automatizadas completas.

Antes de realizar o comando `git push`, Carlos deveria:

- Atualizar sua branch local com as alterações mais recentes do repositório;
- Executar toda a suíte de testes disponível;
- Validar o comportamento da aplicação em um ambiente semelhante ao de integração;
- Corrigir possíveis falhas encontradas;
- Somente então realizar o envio do código para integração.

Esse processo reduz significativamente o risco de introduzir erros na branch principal e mantém a estabilidade do projeto para toda a equipe.

2. A Reação da Equipe

Quando uma pipeline de Integração Contínua falha, a prioridade máxima da equipe deve ser restaurar o estado saudável da aplicação.

O dashboard vermelho do Jenkins indica que a versão atual do código não é considerada confiável. Portanto, nenhuma nova funcionalidade ou correção deve ser integrada até que o problema seja resolvido.

Nesse cenário:

- Carlos deve assumir a investigação inicial da falha, pois sua alteração foi a última integrada;
- Ana deve interromper temporariamente sua correção urgente;
- Toda a equipe deve colaborar para identificar a causa do erro;
- O objetivo principal passa a ser restaurar rapidamente o estado "verde" da pipeline.

Essa prática garante que a branch principal permaneça sempre estável e pronta para novas integrações.

3. O Papel das Ferramentas

Maven

O Maven atuou como ferramenta de automação do processo de build, sendo responsável por:

- Gerenciamento de dependências;
- Compilação do código-fonte;
- Execução de testes automatizados;
- Empacotamento da aplicação;
- Padronização do ciclo de desenvolvimento.

Sua função é garantir que o projeto possa ser construído corretamente de forma reproduzível.

Selenium

O Selenium foi responsável pela execução dos testes funcionais automatizados.

Diferentemente dos testes unitários, ele simula a interação de um usuário real com a interface da aplicação. Dessa forma, conseguiu detectar que o botão "Finalizar Compra" havia desaparecido da tela.

Graças ao Selenium, o defeito foi identificado antes que a aplicação fosse empacotada e disponibilizada para produção, evitando impactos aos clientes.